PBADT504 BIGDATA ANALYTICS REPORT
PROJECT-BASED LEARNING (PBL)

TRAFFICSENSE: URBAN TRAFFIC FLOW ANALYTICS & CONGESTION PREDICTION

Submitted by

Abhijeet Rameshkumar Nair- CEC24AD005

Ananthukrishna CU- CEC24AD014

Abhinav KP- LCEC24AD058

Amarnath A- LCEC24AD061

under the guidance of

Mrs. Shefna Naushad

Assistant Professor

Department of Computer Science and Engineering



COLLEGE OF ENGINEERING, PALLIPPURAM PO, CHERTHALA, ALAPPUZHA, PIN:688541
PHONE:0478 2553416,

APJ Abdul Kalam Technological University [2026-2027]

OCTOBER 2026

Abstract

Traffic congestion is a major urban challenge, and analyzing historical traffic data can help identify recurring traffic patterns and explore congestion prediction. This project, TrafficSense: Urban Traffic Flow Analytics and Congestion Prediction, develops an integrated Big Data Analytics workflow using the Metro Interstate Traffic Volume dataset obtained from Kaggle. The original dataset contains 48,204 records, of which 48,187 valid records were retained for analysis. Python and pandas were used for data preparation and feature engineering, while Hadoop Distributed File System (HDFS) provided storage and Apache Hive supported querying and aggregation. Apache Spark and Spark MLlib were used for descriptive analytics and machine learning experimentation. Traffic patterns were examined across different hours and weather conditions, and the results were visualized using Python charts and a Power BI dashboard. A separate next-hour traffic-category prediction experiment was conducted using the Random Forest algorithm. The model achieved 93.18% accuracy on a chronological historical test set consisting of 7,598 examples, following training on 30,388 examples. The project demonstrates how Big Data technologies can be integrated for historical traffic analysis and prediction experiments. However, it does not currently support live traffic monitoring, and further evaluation is required before real-world deployment.

1. Introduction

1.1 Background

Traffic volume varies according to the time of day, weather conditions, and other contextual factors. Identifying these variations can help us understand recurring traffic patterns and periods of increased congestion. However, manually analyzing large traffic datasets can be difficult and time-consuming.

Big Data Analytics provides tools for storing, processing, querying, and analyzing large datasets efficiently. Technologies such as Hadoop, Hive, and Apache Spark support different stages of this process, while machine learning can be used to explore predictive tasks.

TrafficSense applies these technologies to historical traffic records to analyze traffic volume, examine weather-related patterns, and investigate next-hour congestion-category prediction. Python-based visualizations and a Power BI dashboard are used to present the findings in an understandable format.

1.2 Problem Statement

Historical traffic datasets contain valuable information about traffic volume, timestamps, weather conditions, and other factors. However, extracting meaningful patterns from these records requires systematic data preparation, storage, processing, and analysis.

The main problem addressed by this project is the difficulty of analyzing historical traffic records to identify hourly traffic patterns, compare traffic volume across weather conditions, and investigate the prediction of future congestion categories.

Therefore, the project aims to develop an integrated Big Data Analytics workflow that combines data preprocessing, distributed storage, analytical querying, machine learning experimentation, and visualization.

1.3 Objectives

- The main objectives of the TrafficSense project are:
- To clean and prepare the Metro Interstate Traffic Volume dataset for analysis.
- To analyze historical traffic patterns based on time and weather conditions.
- To use Hadoop HDFS for data storage and Hive for analytical querying.
- To perform descriptive analytics using Apache Spark and PySpark.
- To investigate next-hour traffic-category prediction using the Random Forest algorithm.
- To evaluate the prediction model using a chronological train-test split.
- To visualize analytical results through Python charts and a Power BI dashboard.

1.4 Scope and Significance

The project focuses on historical traffic analysis using Big Data technologies and machine learning. It includes dataset preprocessing, feature engineering, traffic aggregation, weather-based analysis, congestion-category analysis, next-hour prediction experimentation, and dashboard development.

The intended application is the analysis of historical traffic patterns to understand variations in traffic volume and explore the potential of machine learning for congestion prediction.

The project uses the Metro Interstate Traffic Volume dataset, which covers the period from 2012 to 2018. It does not currently incorporate live traffic feeds, GPS-based route information, real-time streaming, route guidance, or automatic traffic-control systems.

The project is significant because it demonstrates how different data technologies can be integrated into a single analytical workflow. It also provides a foundation for future development involving real-time data, additional features, and more comprehensive model evaluation.

2. Literature Review / Existing System

Research in intelligent transportation systems has explored Big Data Analytics, distributed processing, traffic management, and machine learning-based traffic prediction. These approaches demonstrate the potential of computational techniques to analyze traffic information and identify patterns.

The following literature entries were carried forward from the earlier project presentation. Their full bibliographic details should be verified against the original publications before the final report is submitted.

Author / Year	Method or System	Key Finding / Focus	Limitation or Research Gap
Zhu et al. (2019)	Big Data Analytics in intelligent transportation systems	Examines the role of Big Data Analytics in transportation.	Workflow integration was identified as a limitation in the earlier presentation.
Nguyen et al. (2022)	Big Data applications in traffic management	Discusses applications of Big Data in traffic management.	The earlier presentation characterized the analysis as mainly conceptual.
Amini et al. (2017)	Distributed traffic-control analytics	Focuses on distributed architectures for traffic-related analytics.	Limited emphasis on prediction, according to the earlier presentation.
Maarala et al. (2015)	Apache Spark-based traffic-stream analytics	Explores Spark for traffic-stream processing and analytics.	Emphasis on streaming rather than the historical-data workflow used here.
Janković et al. (2021)	Machine learning for traffic-flow prediction	Examines machine learning approaches for traffic prediction.	Requires suitable historical and labelled data.

Existing System

Existing traffic-analysis approaches may focus on individual stages, such as data storage, descriptive analytics, streaming, or machine learning-based prediction. Separate tools and workflows can make it difficult to connect data preparation, analysis, model evaluation, and result visualization.

TrafficSense addresses this integration challenge by combining Python-based preprocessing, HDFS storage, Hive queries, Spark analytics, a Random Forest prediction experiment, and Power BI reporting within one project workflow.

The project does not claim a new prediction algorithm. Its focus is the practical integration of these technologies for historical traffic analysis.

3. Proposed System and Methodology

3.1 Proposed Solution

The proposed TrafficSense system provides an integrated workflow for analyzing historical traffic data and investigating next-hour congestion prediction.

The system begins with the Metro Interstate Traffic Volume dataset. Python and pandas are used to prepare the data and derive useful temporal features. HDFS provides storage, and Hive is used to organize and query traffic records.

Apache Spark performs descriptive analytics, while Spark MLlib supports the Random Forest machine learning experiment. The resulting summaries, prediction results, and visualizations are presented using Python charts and a Power BI dashboard.

The proposed system supports the analysis of hourly traffic variations, weather-based traffic patterns, congestion distributions, and historical next-hour prediction performance.

3.2 System Architecture

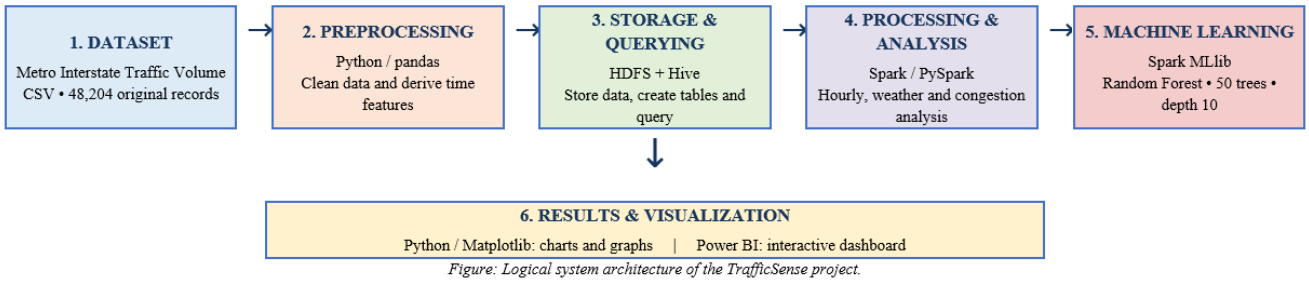


Figure: Logical system architecture of the TrafficSense project.

The system architecture consists of the following components:

1. Data input: The Metro Interstate Traffic Volume CSV dataset serves as the input.
2. Data preprocessing: Python and pandas are used to clean records, parse timestamps, and derive temporal features.
3. Storage: Hadoop HDFS is used to store data files.
4. Querying and aggregation: Hive organizes traffic data into tables and performs analytical queries.
5. Big Data processing: Apache Spark and PySpark perform descriptive analysis and data processing.
6. Machine learning: Spark MLlib is used to train and evaluate a Random Forest model for next-hour traffic-category prediction.
7. Visualization and reporting: Python charts and Power BI present the analytical summaries and model-related distributions.

The overall data flow is:
CSV Dataset → Preprocessing → HDFS / Hive → Spark Analytics → Random Forest Experiment → Charts and Power BI Dashboard

The Spark executor components represent the logical processing architecture and do not imply that a production-scale, multi-node cluster was validated.

3.3 Methodology / Algorithm

The methodology follows a sequence of data preparation, storage, analysis, prediction, and visualization.

Step 1: Data Ingestion
The Metro Interstate Traffic Volume dataset is loaded from a CSV file. It originally contains 48,204 reported records.

Step 2: Data Cleaning and Preprocessing
The dataset is cleaned by filtering an accidental header row, resulting in 48,187 valid analytical records. The timestamp column is parsed, and additional temporal features are derived.

Step 3: Data Storage and Querying
HDFS is used for file storage, and Hive provides structured tables and SQL-like queries for traffic analysis.

Step 4: Descriptive Analytics
Hive and Spark are used to calculate congestion-category counts, hourly traffic averages, and traffic-volume summaries across weather conditions.

Step 5: Next-Hour Dataset Preparation

The observations are sorted chronologically. Only pairs exactly one hour apart are retained for the next-hour prediction task, producing 37,986 examples.

Step 6: Train-Test Split
The data is divided chronologically into 30,388 training examples and 7,598 testing examples. Earlier observations are used for training, and later observations are reserved for testing.

Step 7: Model Training
A Random Forest classifier is trained using Spark MLlib. The model configuration includes 50 trees, a maximum tree depth of 10, and a random seed of 42.
The input features include current traffic volume, temperature, rainfall, snowfall, cloud coverage, hour, day of the week, month, and a weather index.

Step 8: Evaluation and Visualization
The model is evaluated on the held-out historical test data and achieves 93.18% accuracy. Python charts and Power BI are used to communicate the analytical findings.
The accuracy represents historical test performance, not live prediction performance. The earlier classification experiment was affected by target leakage because traffic volume was used as an input feature even though it also defined the congestion label.

3.4 Tools and Technologies

Technology	Version / Type	Purpose
Python	Programming language	Data preparation, scripting, and visualization
pandas	Python library	Data manipulation and preprocessing
Hadoop	3.3.6	Big Data ecosystem
HDFS	Hadoop component	File storage
Hive	3.1.3	Table management and analytical queries
Apache Spark / PySpark	3.5.1	Distributed data processing and analytics
Spark MLlib	Machine learning library	Random Forest model training and evaluation
Matplotlib	Python library	Chart generation
Jupyter Notebook	Development tool	Exploratory analysis
Microsoft Power BI	Visualization platform	Dashboard and analytical reporting
Ubuntu Terminal	Operating environment	Command-line execution
Visual Studio Code	Code editor	Source-code development and editing

4. Dataset / Input Data Description

The project uses the Metro Interstate Traffic Volume dataset obtained from Kaggle.

Dataset source: https://www.kaggle.com/datasets/pooriamst/metro-interstate-traffic-volume

The dataset is supplied as a CSV file and covers the period from 2012 to 2018. It contains timestamp, weather, holiday, and traffic-volume information. The original record count is 48,204; after filtering an accidental header row, 48,187 valid analytical records were retained.

- Dataset name and source/URL: Metro Interstate Traffic Volume https://www.kaggle.com/datasets/pooriamst/metro-interstate-traffic-volume
- Data format: CSV
- Total records/samples: 48,204 reported records
- Number of attributes/features: 9
- Target/output variable : Derived congestion category; next-hour task predicts the category for the following hour

4.1 Feature Description

Field /group	Type/description	Role
date_time	Timestamp of observation	Time index
year, month, day, hour, day_of_week	Derived calendar values	Input features
temp	Temperature	Input feature
rain_1h, snow_1h	Rainfall and snowfall measurements	Input features
clouds_all	Cloud coverage	Input feature
weather_main, weather_description	Recorded weather category and description	Descriptive features
holiday	Holiday context	Context feature
traffic_volume	Measured traffic volume	Analytical variable and next-hour model input
congestion	Category derived from traffic-volume thresholds	Target label

The exact number of attributes is not stated here because a verified schema count was not available in the supplied project details. The congestion thresholds should be documented from the original implementation if exact values are required

4.2 Sample Records and Preprocessing

holiday	temp	rain_1h	snow_1h	clouds_all	weather_m	weather_description	date_time	traffic_volume
None	288.28	0	0	40	Clouds	scattered clouds	02-10-2012 09:00	5545
None	289.36	0	0	75	Clouds	broken clouds	02-10-2012 10:00	4516
None	289.58	0	0	90	Clouds	overcast clouds	02-10-2012 11:00	4767
None	290.13	0	0	90	Clouds	overcast clouds	02-10-2012 12:00	5026
None	291.14	0	0	75	Clouds	broken clouds	02-10-2012 13:00	4918
None	291.72	0	0	1	Clear	sky is clear	02-10-2012 14:00	5181
None	293.17	0	0	1	Clear	sky is clear	02-10-2012 15:00	5584
None	293.86	0	0	1	Clear	sky is clear	02-10-2012 16:00	6015

The preprocessing workflow filtered an accidental header row and retained 48,187 valid analytical records. The timestamp was parsed and temporal features were derived. For next-hour prediction, records were sorted by time and paired only when consecutive observations were exactly one hour apart.

No claim of confirmed duplicate removal is made: the difference between 48,204 reported records and 48,187 retained records is not treated as evidence that all 17 records were duplicates. The report also does not claim a particular missing-value imputation, scaling, or encoding procedure unless it is verified in the implementation.

A raw sample row is not reproduced here because a verified sample record was not included in the source material available for this report. The fields listed in Section 4.1 describe the dataset schema rather than fabricated example values.

5. Implementation

5.1 Module Description

Module	**Responsibility**
Dataset and preprocessing	Load original CSV, clean the dataset, parse timestamps, and derive calendar features.
HDFS and Hive Store files	Organize data into tables, and run SQL-like aggregation queries.
Descriptive analytics	Calculate congestion counts, hourly averages, and weather-based summaries.
Next-hour model	Construct consecutive-hour examples, split chronologically, and train the Random Forest model.
Visualization	Generate Python charts and the Power BI dashboard.
Validation and reporting	Check record counts, split sizes, recorded model accuracy, and report limitations.

5.2 Source Code and Execution

File / artifact	**Purpose**
01_Dataset/Metro_Interstate_Traffic_Volume.csv	Original input dataset.
01_Dataset/traffic_cleaned.csv	Prepared analytical dataset.

02_Source_Code/first_stage.ipynb	Notebook used for early-stage analysis.
02_Source_Code/spark_hive_analysis.py	Reconstructed script for Spark/Hive analysis; validate before reuse.
02_Source_Code/create_visualizations.py	Reconstructed chart-generation script; validate before reuse.
02_Source_Code/train_next_hour_model.py	Reconstructed next-hour model script; validate before reuse.
03_Hive_SQL/analysis_reference.sql	Reference SQL queries; validate against the configured Hive schema.
04_Results/next_hour_forecast_results.csv	Saved forecast results.
04_Results/*.png and supporting CSV	Saved analytical charts and feature-importance artifacts.
05_Dashboard/visualization.pbix	Power BI dashboard file.

5.3 Execution Procedure

• Place the original CSV and cleaned dataset in the project dataset folder.
• Start and verify Hadoop/HDFS services in the configured environment.
• Upload or verify the data files in HDFS, then confirm that the Hive database and tables exist.
• Run the applicable Hive queries to produce congestion, hourly, and weather summaries.
• Run the Spark analysis and next-hour model workflow with the required Python, Spark, and Hive configuration.
• Inspect the saved CSV results and generated charts.
• Open the Power BI .pbix file and verify that its data sources and visuals are available.

Exact commands and environment-specific configuration should be taken from the validated setup. Service availability and paths must be checked during execution; saved outputs are available as a demonstration fallback.

6. Testing and Results

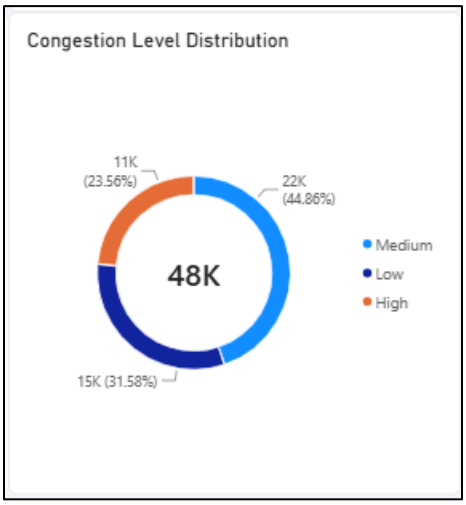
6.1 Testing

Test ID	Scenario	Expected / check	Recorded outcome
T1	Dataset preparation	Retain valid analytical records	48,187 records retained
T2	Hive congestion aggregation	Count records by category	High 11,353; Low 15, 216; Medium 21,618
T3	Hourly analysis	Produce hourly summaries	Hourly averages available
T4	Weather analysis	Summarize traffic by weather condition	Summary available
T5	Next-hour pair construction	Keep observations exactly one hour apart	37, 986 examples

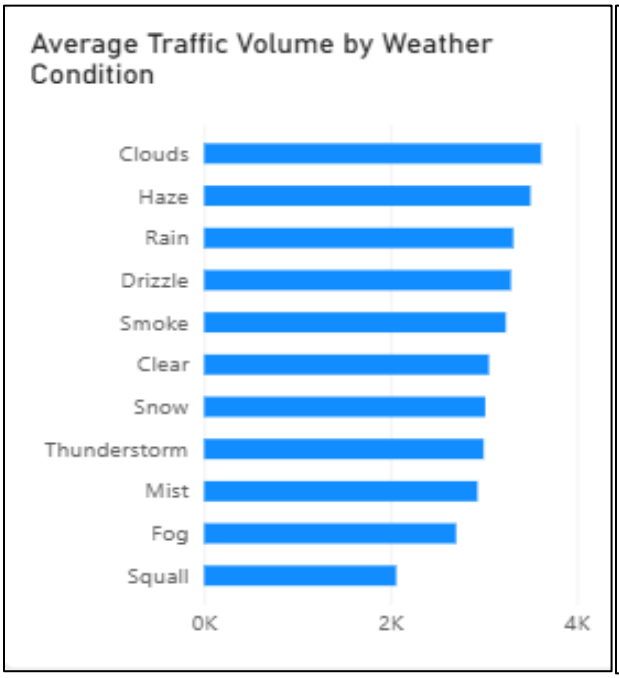
T6	Chronological split	Preserve temporal order	30, 388 train; 7, 598 test
T7	Model evaluation	Evaluate held-out test examples	93.18% historical accuracy
T8	Dashboard	Display summaries and distributions	Power BI dashboard file recorded

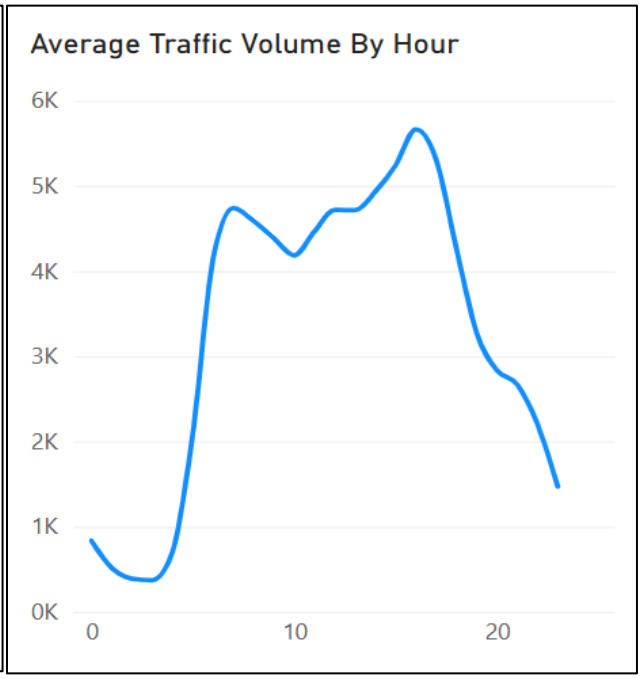
The table records outcomes documented for the project. It does not imply that all services were re-executed during preparation of this report.

6.2 Results

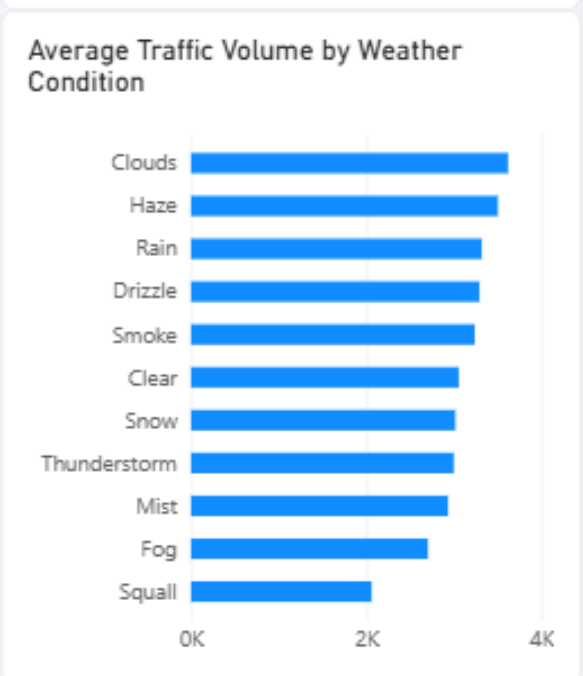


The congestion aggregation produced 11,353 High, 15,216 Low, and 21,618 Medium records, totalling 48,187. The recorded average traffic volumes by category were approximately 5,751.82 for High, 3,685.86 for Medium, and 794.55 for Low.

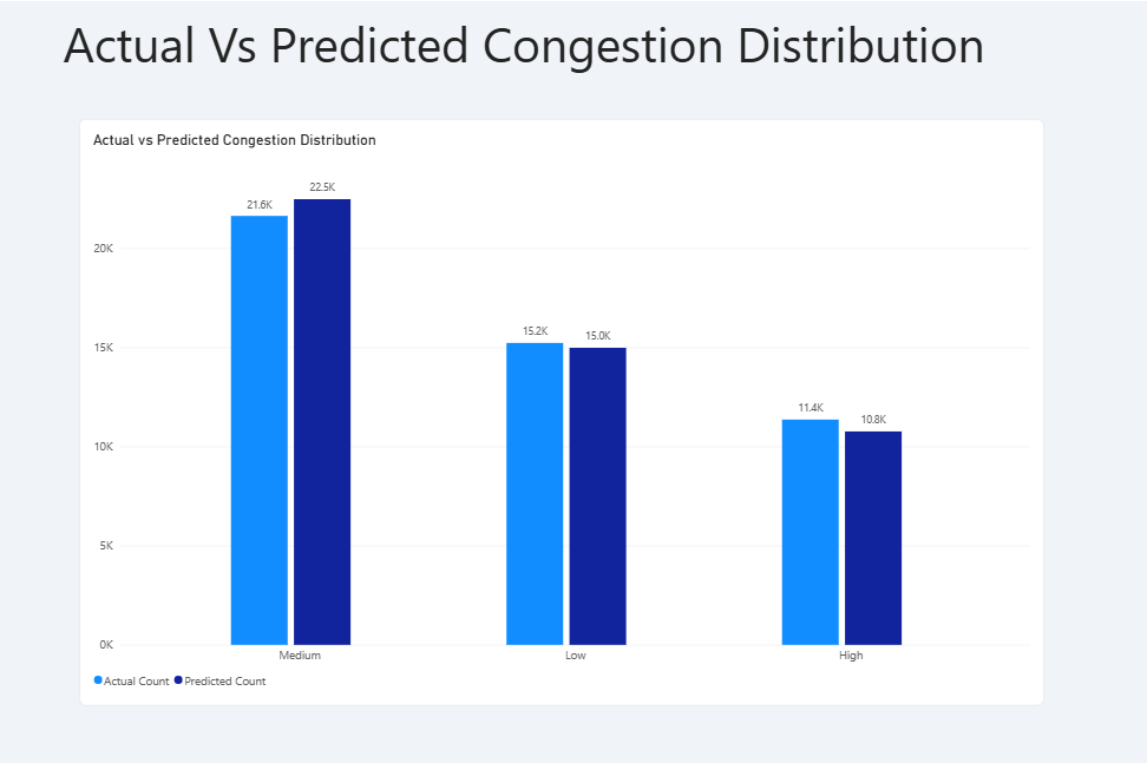




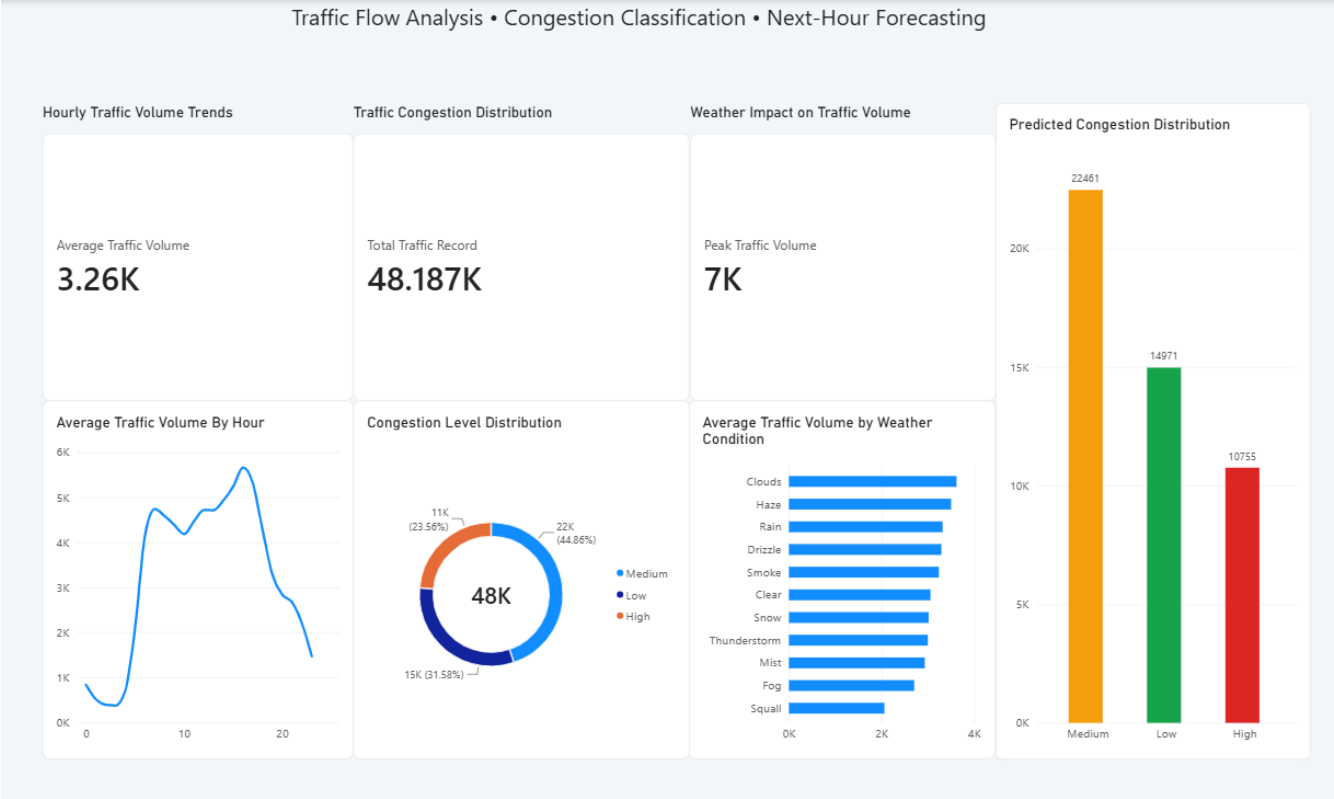
Hourly analysis highlighted average traffic volumes of approximately 5,240.46 at 3 PM, 5,663.76 at 4 PM, and 5,310.08 at 5 PM. The dataset-wide average traffic volume was approximately 3.26K, with a peak of around 7K in the recorded dashboard summary



The weather analysis compared average traffic volume across recorded weather categories. These are descriptive associations and do not establish that weather alone causes traffic changes



The next-hour Random Forest experiment used 37,986 consecutive-hour examples, with 30,388 for training and 7,598 for testing. The chronological boundary was recorded as 2017-11-16 20:00. The model used 50 trees, maximum depth 10, and seed 42, and achieved 93.18% accuracy on the held-out historical test set.



The Power BI dashboard compares actual and predicted category distributions: Low 15,216 actual versus 14,971 predicted; Medium 21,618 actual versus 22,461 predicted; and High 11,353 actual versus 10,755 predicted. These values compare totals only and are not a per-record confusion matrix.

6.3 Performance Evaluation

The reported model metric is 93.18% accuracy on a chronological held-out historical test set of 7,598 examples. The data was split by time, with earlier examples used for training and later examples reserved for testing. Precision, recall, F1-score, and a confusion matrix were not available in the recorded results and are not reported here.

An earlier classification experiment was affected by target leakage because traffic_volume was used as a feature even though it also defined the congestion label. Its score is therefore not treated as reliable evidence of future prediction performance. The 93.18% result refers to the separate next-hour task and is not a live deployment metric. The dataset and current workflow also do not provide route-level GPS information or live traffic feeds.

7. Individual Contribution and Teamwork

Team member	Assigned role/work recorded
Abhijeet Rameshkumar Nair (CEC24AD005)	Primary technical implementation, system architecture, project integration, data processing and feature engineering
Ananthukrishna CU (CEC24AD014)	Literature survey and dataset-selection support; presentation preparation and documentation support
Abhinav KP (LCEC24AD058)	Visualization, dashboard development, analysis support, and testing assistance.
Amarnath A (LCEC24AD061)	Testing support, documentation, and project-review assistance.

The team collaborated on dataset selection, architecture reviews, project integration, and presentation milestones. Responsibilities are represented according to the roles recorded in the project presentation; they do not imply identical coding responsibilities

8. Conclusion and Future Scope

8.1 Conclusion

TrafficSense demonstrates an integrated Big Data Analytics workflow for historical traffic analysis. The project retained 48,187 valid analytical records, used Hive and Spark to produce congestion, hourly, and weather summaries, and presented results through Python charts and a Power BI dashboard. A separate next-hour Random Forest experiment achieved 93.18% accuracy on a chronological historical test set of 7,598 examples after training on 30,388 examples. These results show the practical integration of data preparation, storage, analytics, machine learning, and reporting. The system is an offline historical-data project rather than a live traffic service.

8.2 Limitations and Future Enhancements

- The project uses historical observations and does not ingest live traffic feeds.
- The dataset does not provide the detailed road-network and GPS information needed for route-specific predictions.
- The 93.18% accuracy is a historical test result and does not guarantee live performance.
- Precision, recall, F1-score, and a confusion matrix should be computed from verified per-record test predictions in future evaluation.
- The earlier classifier had target leakage; the feature/target pipeline should be reviewed carefully.
- Future work can compare baseline models, incorporate broader data and geographic information, explore sensors/GPS and streaming technologies such as Kafka, and validate deployment only after further testing.

References

[1] Metro Interstate Traffic Volume dataset, Kaggle. https://www.kaggle.com/datasets/pooriamst/metro-interstate-traffic-volume

[2] Apache Hadoop Documentation. https://hadoop.apache.org/docs/

[3] Apache Hive Documentation. https://hive.apache.org/

[4] Apache Spark Documentation, including MLlib. https://spark.apache.org/docs/latest/

[5] Microsoft Power BI Documentation. https://learn.microsoft.com/power-bi/

[6] Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

[7] Zhu et al. (2019); Nguyen et al. (2022); Amini et al. (2017); Maarala et al. (2015); Jankovi■ et al. (2021). Literature-review entries carried forward from the earlier project presentation; complete bibliographic metadata requires verification against the original papers.

Appendix A: Source Code and Project Files

https://github.com/abhinair302/Big-Data-Analytics-Project-Semester-5

Project structure

```
TrafficSense_Project/
|-- 01_Dataset/
|   |-- Metro_Interstate_Traffic_Volume.csv
|   `-- traffic_cleaned.csv
|-- 02_Source_Code/
|   |-- first_stage.ipynb
|   |-- spark_hive_analysis.py
|   |-- create_visualizations.py
|   |-- train_next_hour_model.py
|   `-- README.md
|-- 03_Hive_SQL/
|   |-- analysis_reference.sql
|   `-- README.md
|-- 04_Results/
|   |-- congestion_distribution.png
|   |-- feature_importance.png
|   |-- feature_importance_without_volume.csv
|   |-- next_hour_forecast_results.csv
|   |-- traffic_by_hour.png
|   `-- weather_impact.png
|-- 05_Dashboard/hello.pbix
|-- 06_Documentation/
|   |-- artifact_checklist.md
|   `-- presentation_notes.md
|-- README.md
`-- README_backup.md
```

Appendix B: Additional Screenshots / Outputs

Hive Querying:

```
abhijeet@LAPTOP-VBFS4NSG:~$ hive -e "SELECT congestion, COUNT(*) AS record_count FROM traffic_db.traffic_data WHERE congestion IN ('Low','Medium','High') GROUP BY congestion ORDER BY congestion;"
Total MapReduce CPU Time Spent: 0 msec
OK
High    11353
Low     15216
Medium  21618
Time taken: 17.49 seconds, Fetched: 3 row(s)
```

```
abhijeet@LAPTOP-VBFS4NSG:~$ hive -e "SELECT hour, ROUND(AVG(traffic_volume), 2) AS avg_traffic FROM traffic_db.traffic_data GROUP BY hour ORDER BY hour;"
```

```
Total MapReduce CPU Time Spent: 0 msec
OK
0       834.98
1       516.34
2       388.35
3       371.09
4       702.55
5       2094.57
6       4140.5
7       4740.18
8       4587.5
9       4385.28
10      4184.14
11      4465.57
12      4718.03
13      4714.65
14      4931.89
15      5240.46
16      5663.76
17      5310.08
18      4263.56
19      3275.21
20      2834.67
21      2668.94
22      2199.41
```

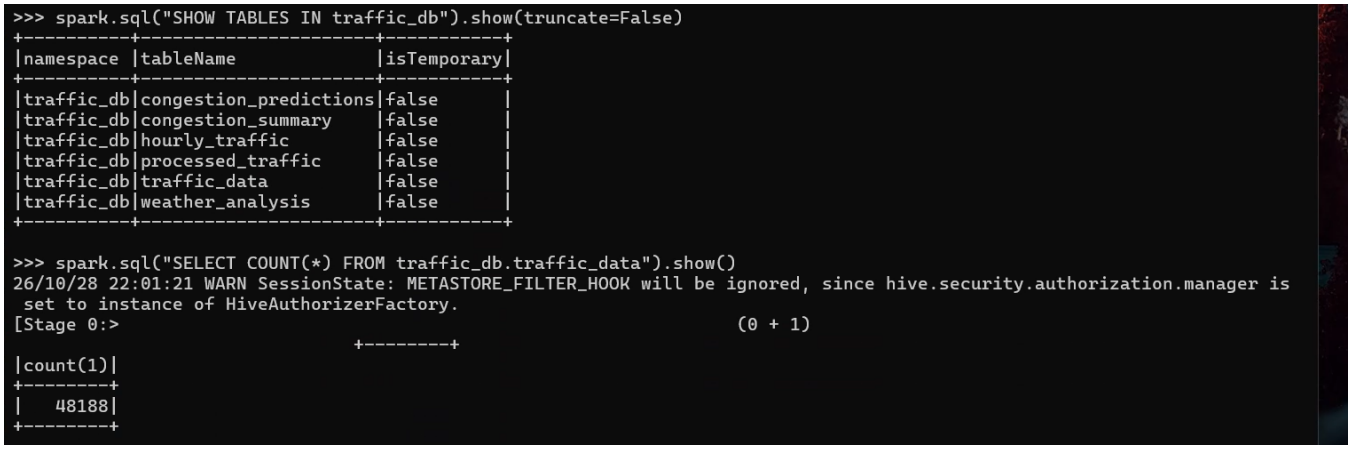
HDFS:

```
abhijeet@LAPTOP-VBFS4NSG:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [LAPTOP-VBFS4NSG]
abhijeet@LAPTOP-VBFS4NSG:~$ jps
1204 Jps
709 DataNode
921 SecondaryNameNode
604 NameNode
```

Spark:

```
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.5.1
      /_/

Using Python version 3.10.12 (main, Aug 31 2026 10:18:17)
Spark context Web UI available at http://10.255.255.254:4040
Spark context available as 'sc' (master = local[*], app id = local-1793205013757).
SparkSession available as 'spark'.
>>> spark.sql("SHOW DATABASES").show(truncate=False)
26/10/28 22:00:40 WARN HiveConf: HiveConf of name hive.metastore.event.db.notification.api.auth does not exist
+-----------+
|namespace  |
+-----------+
|default    |
|traffic_db|
+-----------+
```


>>> spark.sql("SHOW TABLES IN traffic_db").show(truncate=False)
+---------+---------------------+-----------+
|namespace |tableName |isTemporary|
+---------+---------------------+-----------+
traffic_db	congestion_predictions	false
traffic_db	congestion_summary	false
traffic_db	hourly_traffic	false
traffic_db	processed_traffic	false
traffic_db	traffic_data	false
traffic_db	weather_analysis	false
+---------+---------------------+-----------+
>>> spark.sql("SELECT COUNT(*) FROM traffic_db.traffic_data").show()
26/10/28 22:01:21 WARN SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is
set to instance of HiveAuthorizerFactory.
[Stage 0:> (0 + 1)
+--------+
|count(1)|
+--------+
| 48188|
+--------+